MASTER TECHNICAL SPECIFICATION & ARCHITECTURE

Smart Library & Study Center Management System

Master Technical Architecture & Engineering Specification

Document Reference	LMS-SPEC-V1.0
Architecture Version	1.0.0 (Pre-Implementation Baseline)
Classification	Technical Source of Truth (SSOT)
Effective Date	October 2026
Facility Scale	100 Physical Seats (Desk 1 to 100)
Daily Capacity	300 Seat-Slots / 1,600 Daily Seat-Hours
Operating Hours	07:00 AM – 11:00 PM (16 Operating Hours / Day)
Primary Core Stack	Python 3.10+ | FastAPI | PostgreSQL 15+ | SQLAlchemy 2.0 | Streamlit
Authorial Roles	Senior Software Architect, Backend Engineer, Database Architect, Data Engineer

CONFIDENTIAL & PROPRIETARY

This master architecture document governs the end-to-end engineering, database constraints, business logic, API contracts, and phased execution of the Smart Library & Study Center platform. Autonomous AI models and human developers must adhere to this document as the authoritative architectural baseline.

Smart Library & Study Center Management System

Master Technical Architecture & Specification

Document Reference: LMS-SPEC-V1.0
Document Classification: Technical Architecture & Engineering Source of Truth
Version: 1.0.0
Effective Date: October 2026
Status: Architecture Approved / Pre-Implementation
Target Infrastructure: 100 Physical Seats (Desk 1 to 100)
Operating Schedule: 07:00 AM – 11:00 PM Daily (16 Operating Hours / Day)
Core Stack: Python 3.10+, FastAPI, PostgreSQL 15+, SQLAlchemy 2.0, Pydantic v2, Streamlit

Executive Summary & Architecture Metadata

Specification Key	Definition / Value
System Name	Smart Library & Study Center Management System
Authorial Role	Senior Software Architect, Backend Engineer, Database Architect & Data Engineer
Primary Audience	Senior Engineers, Autonomous AI Agents, Database Administrators, Product Owners
Physical Inventory	100 Dedicated Study Desks ($1 \rightarrow 100$)
Daily Shift Inventory	4 Canonical Shifts: Morning (7-11), Afternoon (11-15), Evening (15-23), Full Day (7-23)
Daily Slot Capacity	300 Shift-Slots / Day (or 1,600 Daily Seat-Hours)
Concurrency Contract	Zero double-bookings guaranteed at database engine level via GiST multidimensional exclusion
Presentation Contract	Presentation layer (Streamlit V1) is decoupled; communicates exclusively via HTTP REST

Table of Contents

1. Project Overview
2. Business Requirements
3. Business Rules
4. Membership Lifecycle
5. Seat/Shift Allocation Logic
6. Database Architecture
7. Final PostgreSQL Schema
8. Entity-Relationship (ER) Diagram
9. Application Architecture
10. Folder Structure
11. API Architecture & REST Endpoint Reference
12. Service & Business Logic Algorithms
13. Payment System & Financial Ledger
14. Renewal System & Chain Lineage
15. Attendance System & Check-in Rules
16. Dashboard, Metrics & Analytics Engine
17. Security Architecture & Threat Mitigation
18. Testing Strategy & Pytest Verification Matrix
19. Real-World Edge Cases & Handling Matrix

20. Backup, Disaster Recovery & Data Portability
21. Future Data Science & Machine Learning Architecture
22. Phased Development Roadmap
23. Architecture Decisions and Rationale (ADRs)
24. Known Limitations of Version 1.0
25. Future Expansion & Enterprise Roadmap

---

1. Project Overview

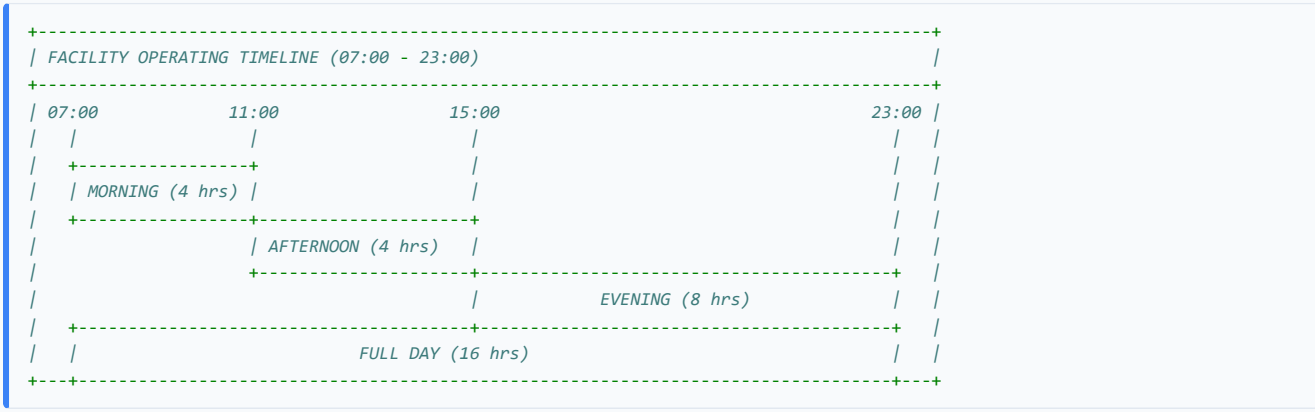
1.1 Context & Real-World Domain

Modern study centers and reading rooms operate on a space-as-a-service model tailored for students and aspirants preparing for competitive examinations (e.g., UPSC, GATE, Medical, CA Foundation). Unlike traditional lending libraries that manage physical book collections, study centers monetize **quiet physical desk infrastructure**. Desks are equipped with high-speed Wi-Fi, power outlets, reading lamps, ergonomic chairs, and optional storage lockers.

Because commercial real estate incurs fixed rent per square foot, study centers rely on **temporal multiplexing of physical inventory**: leasing the exact same physical desk to different students across non-overlapping shifts throughout the day.

1.2 Capacity & Facility Timeline

- **Physical Desks**: Exactly 100 physical study seats numbered sequentially from `1` to `100`.
- **Facility Operating Hours**: `07:00 AM` to `11:00 PM` daily (16 operating hours per day, 7 days a week).
- **Canonical Daily Shifts**:
  1. `MORNING`: `07:00 AM – 11:00 AM` (4 hours)
  2. `AFTERNOON`: `11:00 AM – 03:00 PM` (4 hours)
  3. `EVENING`: `03:00 PM – 11:00 PM` (8 hours)
  4. `FULL_DAY`: `07:00 AM – 11:00 PM` (16 hours)



1.3 Core Engineering Challenge

In manual or poorly architected spreadsheet systems, desk management suffers from high collision rates:

- Assigning Rahul and Amit to Seat 25 on overlapping shifts.
- Selling Full Day to Suresh on Seat 25 when Rahul already holds Morning shift.
- Permitting expired students to occupy chairs unnoticed.
- Financial balance drift across renewal cycles.

This system establishes a deterministic, mathematically validated management platform where physical and temporal invariants are enforced **strictly at the database and service layers**.

---

2. Business Requirements

2.1 Functional Requirements (FR)

- **FR-01: Seat Inventory Management**: Track 100 physical seats, their operational status (`ACTIVE` vs `MAINTENANCE`), and physical amenities (power outlet, locker).
- **FR-02: Student Profile Directory**: Maintain unique student records indexed by mobile number, identity proof, and emergency contacts.
- **FR-03: Temporal Seat Allocation Engine**: Allocate seats to students across date ranges and shift hours with absolute collision detection.
- **FR-04: Subscription Lifecycle Management**: Support atomic student admission, renewal chaining, premature cancellation, and seat release.
- **FR-05: Ledger-Based Financial Accounting**: Record membership billing targets, handle installment/partial payments, issue receipts, and manage payment voiding with audit logs.
- **FR-06: Daily Attendance Tracking**: Log check-ins and check-outs validated against active memberships within operating shift windows.
- **FR-07: Operational Dashboards & Analytics**: Provide real-time visibility into seat slot occupancy, expiring memberships, and daily financial collections.

2.2 Non-Functional Requirements (NFR)

- **NFR-01: Absolute Double-Booking Prevention**: Mathematical impossibility of two active memberships sharing the same seat, date, and time.
- **NFR-02: Sub-10ms Allocation Performance**: Fast conflict checking via indexed multidimensional range trees (`btree_gist`).
- **NFR-03: Zero Stale State via Dynamic Derivation**: Time-dependent statuses (`EXPIRING_SOON`, `EXPIRED`) must be computed dynamically, eliminating midnight cron failure vulnerabilities.
- **NFR-04: Strict Audit Trail**: All financial cancellations, voids, and administrative changes must be permanently recorded in an immutable audit table.
- **NFR-05: Decoupled Architecture**: Full presentation decoupling so the Streamlit frontend can be replaced by a React web application or mobile app with zero backend modification.

---

3. Business Rules

3.1 Physical & Human Invariants

1. **Rule BR-01 (Single Physical Human Rule)**: A single student cannot occupy two physical seats at the exact same moment in time. A student holding a Morning shift on Seat 10 cannot book a Morning shift on Seat 20 on the same date.
2. **Rule BR-02 (Multi-Shift Eligibility)**: A single student MAY hold two distinct memberships across the same calendar date, provided their daily time intervals are completely disjoint (e.g., Morning on Seat 25 and Evening on Seat 40).
3. **Rule BR-03 (Full Day Exclusivity)**: A Full Day membership ($07:00 \rightarrow 23:00$) on a seat completely consumes that seat for the entire calendar date range, blocking Morning, Afternoon, and Evening shifts.
4. **Rule BR-04 (Consecutive Shift Boundary Rule)**: Shifts ending and starting at the exact same hour (e.g., Morning ending at 11:00 AM and Afternoon starting at 11:00 AM) **do not collide**. Shifts operate as half-open intervals $[T_{\text{start}}, T_{\text{end}})$.
5. **Rule BR-05 (Seat Inactivity)**: A seat marked `is_active = FALSE` (under maintenance/damaged) cannot be assigned to any new membership.
6. **Rule BR-06 (Student Status vs Membership Status)**: A student's profile status (`ACTIVE`, `INACTIVE`, `BLOCKED`) is independent of whether they currently possess an active membership. Expired students remain `ACTIVE` in the student registry.

---

4. Membership Lifecycle

4.1 Persisted vs Derived States: Eliminating the Flaw

Architectural Analysis

The original draft proposed persisting `EXPIRING_SOON` and `EXPIRED` directly inside a database column. This introduces severe engineering flaws:

1. **The Stale State Bug**: If a student's membership expires at midnight, but a server reboot or cron crash delays the update script, the database falsely evaluates `WHERE status = 'ACTIVE'`, granting illicit entry or blocking re-allocation.

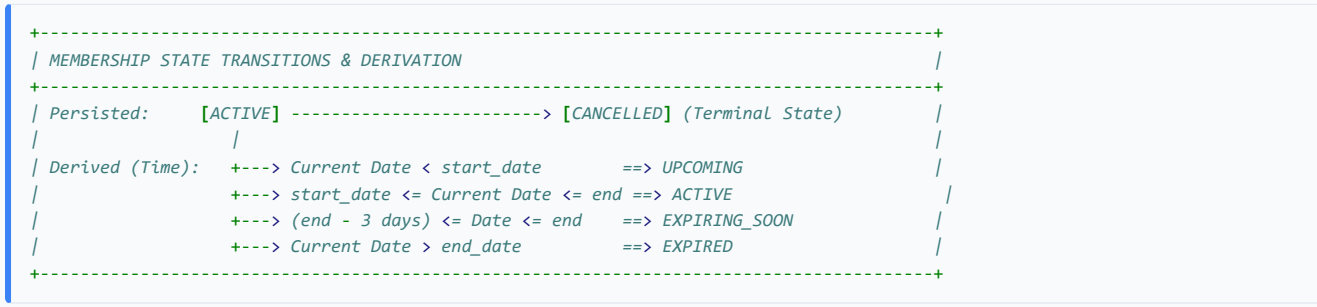
2. **Write Amplification & Table Bloat**: Nightly updates of thousands of historical rows generate massive Write-Ahead Log (WAL) traffic, trigger table vacuum churn, and rewrite database index pages.
3. **Timezone Skew**: Statically written expiry flags introduce ambiguities across server and client timezones.

Architectural Standard

1. **Persisted State (`memberships.status`)**: Only explicit operational lifecycle transitions are stored on disk:
   - `ACTIVE` : The membership contract is legally in effect and active.
   - `CANCELLED` : The membership was prematurely terminated by administrative action.
2. **Derived Temporal State (Virtual State Machine)**: The operational status presented to users and APIs is computed dynamically on read:

$$\text{DerivedStatus}(m, t) = \begin{cases} \text{CANCELLED} & \text{if } m.\text{status} = \text{'CANCELLED'} \\ \text{UPCOMING} & \text{if } m.\text{status} = \text{'ACTIVE'} \land t < m.\text{start_date} \\ \text{EXPIRED} & \text{if } m.\text{status} = \text{'ACTIVE'} \land t > m.\text{end_date} \\ \text{EXPIRING_SOON} & \text{if } m.\text{status} = \text{'ACTIVE'} \land t \le m.\text{end_date} \land t \ge (m.\text{end_date} - \Delta_{\text{alert}}) \\ \text{ACTIVE} & \text{if } m.\text{status} = \text{'ACTIVE'} \land t \ge m.\text{start_date} \land t \le m.\text{end_date} \end{cases}$$

Where $\Delta_{\text{alert}} = 3 \text{ days}$.



---

5. Seat/Shift Allocation Logic

5.1 Temporal Representation: Half-Open Intervals

To eliminate boundary collision bugs, daily operating shifts are defined as **half-open time intervals** $[T_{\text{start}}, T_{\text{end}})$:

- Shift 1: $[07:00, 11:00)$ — The student's entitlement terminates at `10:59:59.999` .
- Shift 2: $[11:00, 15:00)$ — The student's entitlement begins at `11:00:00.000` .

In integer minutes from midnight ($0 \le m < 1440$):

- `MORNING` : $[420, 660)$
- `AFTERNOON` : $[660, 900)$
- `EVENING` : $[900, 1380)$
- `FULL_DAY` : $[420, 1380)$

5.2 Mathematical Overlap Formula

Two time intervals $[S_1, E_1)$ and $[S_2, E_2)$ collide if and only if:

$$\max(S_1, S_2) < \min(E_1, E_2) \iff S_1 < E_2 \land S_2 < E_1$$

5.3 Calendar Date Convention: Inclusive Days $[D_{\text{start}}, D_{\text{end}}]$

Unlike daily times, calendar dates in commercial library billing represent **inclusive calendar days**:

- When a student subscribes for October: `start_date = 2026-10-01` , `end_date = 2026-10-31` . They have full access on October 31st.
- A renewal subscription starts on: `start_date = 2026-11-01` , `end_date = 2026-11-30` .

- Two inclusive date ranges $[S_1, E_1]$ and $[S_2, E_2]$ collide if and only if:

$$S_1 \leq E_2 \wedge S_2 \leq E_1$$

- For Oct 1–31 and Nov 1–30: $2026\text{-}10\text{-}01 \leq 2026\text{-}11\text{-}30$ (True), BUT $2026\text{-}11\text{-}01 \leq 2026\text{-}10\text{-}31$ (False). **Result: Zero Overlap**. Seamless renewal without false collision.

5.4 Conflict Evaluation Scenarios Matrix

Allocation 1	Allocation 2	Start 1 / End 1	Start 2 / End 2	Condition $S_1 < E_2 \wedge S_2 < E_1$	System Action	Engineering Rationale
Morning	**Afternoon**	07:00 / 11:00	11:00 / 15:00	$07:00 < 15:00 \wedge 11:00 < 11:00$ (False)	**ALLOW**	Touching boundaries; zero instantaneous overlap.
Afternoon	**Evening**	11:00 / 15:00	15:00 / 23:00	$11:00 < 23:00 \wedge 15:00 < 15:00$ (False)	**ALLOW**	Touching boundaries; clean shift handover.
Morning	**Full Day**	07:00 / 11:00	07:00 / 23:00	$07:00 < 23:00 \wedge 07:00 < 11:00$ (True)	**REJECT**	Full Day completely engulfs Morning shift hours.
Afternoon	**Full Day**	11:00 / 15:00	07:00 / 23:00	$11:00 < 23:00 \wedge 07:00 < 15:00$ (True)	**REJECT**	Full Day blocks afternoon hours.
Evening	**Full Day**	15:00 / 23:00	07:00 / 23:00	$15:00 < 23:00 \wedge 07:00 < 23:00$ (True)	**REJECT**	Full Day blocks evening hours.
Morning	**Custom 9-1**	07:00 / 11:00	09:00 / 13:00	$07:00 < 13:00 \wedge 09:00 < 11:00$ (True)	**REJECT**	Overlap between 09:00 and 11:00.
Seat 25	**Seat 26**	07:00 / 11:00	07:00 / 11:00	Different `seat_id`	**ALLOW**	Physical seats are distinct; identical hours permitted.

---

6. Database Architecture

6.1 Database Engine & Relational Integrity

- **Database Engine**: PostgreSQL 15+ Enterprise Relational Database.
- **Normalization**: Normalized to Third Normal Form (3NF) for operational transactions, with SQL Views providing denormalized projections for reporting.
- **ACID Transaction Isolation**: Default `READ COMMITTED` with explicit Pessimistic Locking (`FOR UPDATE`) on seat allocation.
- **Physical Exclusion via `btree_gist`**: Utilizing PostgreSQL Generalized Search Tree (GiST) indexing over composite scalars (`seat_id =`) and ranges (`daterange &&` , `int4range &&`).

6.2 The Multidimensional Exclusion Constraint Explained

In PostgreSQL, standard `UNIQUE` constraints only test scalar equality. A temporal booking requires testing **equality on seat ID** combined with **two-dimensional range intersection on dates and minutes**:

```
CONSTRAINT exclude_seat_time_conflict EXCLUDE USING gist (
    seat_id WITH =,
    daterange(start_date, end_date, '[]') WITH &&,
    int4range(start_minute, end_minute, '[)') WITH &&
) WHERE (status = 'ACTIVE');
```

When an `INSERT` or `UPDATE` executes, PostgreSQL checks the GiST R-Tree index. If any existing active row shares the exact same `seat_id` , overlaps the requested `daterange` , and overlaps the requested `int4range` , the transaction is aborted by PostgreSQL's storage engine. This provides mathematical immunity against application bugs and race conditions.

---

7. Final PostgreSQL Schema

```sql
-- ============================================================================
-- SMART LIBRARY & STUDY CENTER MANAGEMENT SYSTEM
-- MASTER PRODUCTION DDL SPECIFICATION (POSTGRESQL 15+)
-- ============================================================================

-- Extensions
CREATE EXTENSION IF NOT EXISTS "btree_gist";
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";

-- Domain Enums
CREATE TYPE user_role_enum AS ENUM ('ADMIN', 'STAFF');
CREATE TYPE student_status_enum AS ENUM ('ACTIVE', 'INACTIVE', 'BLOCKED');
CREATE TYPE membership_status_enum AS ENUM ('ACTIVE', 'CANCELLED');
CREATE TYPE payment_method_enum AS ENUM ('CASH', 'UPI', 'CARD', 'NET_BANKING');
CREATE TYPE payment_status_enum AS ENUM ('COMPLETED', 'VOIDED');
CREATE TYPE attendance_status_enum AS ENUM ('PRESENT', 'ABSENT', 'HALF_DAY');
CREATE TYPE attendance_source_enum AS ENUM ('MANUAL_STAFF', 'KIOSK', 'BIOMETRIC_SYNC');

-- ============================================================================
-- 1. USERS & STAFF AUTHENTICATION
-- ============================================================================
CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    username VARCHAR(50) NOT NULL UNIQUE,
    email VARCHAR(255) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    full_name VARCHAR(100) NOT NULL,
    role user_role_enum NOT NULL DEFAULT 'STAFF',
    is_active BOOLEAN NOT NULL DEFAULT TRUE,
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_users_username ON users(username);
CREATE INDEX idx_users_email ON users(email);

-- ============================================================================
-- 2. STUDENTS DIRECTORY
-- ============================================================================
CREATE TABLE students (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    student_code VARCHAR(20) NOT NULL UNIQUE, -- E.g. STU-2026-0001
    full_name VARCHAR(100) NOT NULL,
    mobile VARCHAR(15) NOT NULL UNIQUE,
    email VARCHAR(255) NULL,
    id_proof_type VARCHAR(50) NULL,           -- AADHAAR, PAN, DRIVING_LICENSE
    id_proof_number VARCHAR(100) NULL,
    emergency_contact VARCHAR(15) NULL,
    address TEXT NULL,
    status student_status_enum NOT NULL DEFAULT 'ACTIVE',
    notes TEXT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_students_mobile ON students(mobile);
CREATE INDEX idx_students_status ON students(status);
CREATE INDEX idx_students_full_name ON students(full_name);

-- ============================================================================
-- 3. PHYSICAL SEATS INVENTORY (1 TO 100)
-- ============================================================================
CREATE TABLE seats (
    id INT PRIMARY KEY,                       -- Exactly 1 to 100
    seat_number INT NOT NULL UNIQUE CHECK (seat_number >= 1 AND seat_number <= 100),
    is_active BOOLEAN NOT NULL DEFAULT TRUE,  -- False if desk damaged/in maintenance
    has_power_outlet BOOLEAN NOT NULL DEFAULT TRUE,
    has_locker BOOLEAN NOT NULL DEFAULT FALSE,
```

```
    description VARCHAR(255) NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_seats_is_active ON seats(is_active);

-- ============================================================================
-- 4. OPERATING SHIFTS
-- ============================================================================
CREATE TABLE shifts (
    id SERIAL PRIMARY KEY,
    name VARCHAR(50) NOT NULL UNIQUE,          -- MORNING, AFTERNOON, EVENING, FULL_DAY
    start_time TIME NOT NULL,                  -- 07:00:00
    end_time TIME NOT NULL,                    -- 11:00:00
    start_minute INT NOT NULL,                 -- 420
    end_minute INT NOT NULL,                   -- 660
    base_fee NUMERIC(10, 2) NOT NULL CHECK (base_fee >= 0),
    is_active BOOLEAN NOT NULL DEFAULT TRUE,
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT chk_shift_time_order CHECK (start_time < end_time),
    CONSTRAINT chk_shift_minute_order CHECK (start_minute < end_minute AND start_minute >= 0 AND end_minute <= 1440)
);

-- ============================================================================
-- 5. MEMBERSHIPS (SUBSCRIPTION CONTRACTS)
-- ============================================================================
CREATE TABLE memberships (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    student_id UUID NOT NULL REFERENCES students(id) ON DELETE RESTRICT,
    seat_id INT NOT NULL REFERENCES seats(id) ON DELETE RESTRICT,
    shift_id INT NOT NULL REFERENCES shifts(id) ON DELETE RESTRICT,

    start_date DATE NOT NULL,
    end_date DATE NOT NULL,

    -- Canonical time bounds copied from shift at creation time
    start_minute INT NOT NULL,
    end_minute INT NOT NULL,

    -- Financial contract
    base_fee NUMERIC(10, 2) NOT NULL CHECK (base_fee >= 0),
    discount_amount NUMERIC(10, 2) NOT NULL DEFAULT 0.00 CHECK (discount_amount >= 0),
    final_fee NUMERIC(10, 2) NOT NULL CHECK (final_fee >= 0),

    -- Persisted Lifecycle State
    status membership_status_enum NOT NULL DEFAULT 'ACTIVE',

    -- Cancellation metadata
    cancelled_at TIMESTAMPTZ NULL,
    cancelled_by UUID NULL REFERENCES users(id) ON DELETE SET NULL,
    cancel_reason TEXT NULL,

    -- Renewal chain link
    renewed_from_id UUID NULL REFERENCES memberships(id) ON DELETE SET NULL,

    created_by UUID NULL REFERENCES users(id) ON DELETE SET NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT chk_membership_date_order CHECK (start_date <= end_date),
    CONSTRAINT chk_membership_final_fee CHECK (final_fee = (base_fee - discount_amount)),
    CONSTRAINT chk_membership_cancelled_meta CHECK (
        (status = 'CANCELLED' AND cancelled_at IS NOT NULL AND cancel_reason IS NOT NULL)
        OR (status = 'ACTIVE' AND cancelled_at IS NULL AND cancel_reason IS NULL)
    ),

    -- MULTIDIMENSIONAL EXCLUSION CONSTRAINT 1: PREVENTS SEAT DOUBLE BOOKING
    CONSTRAINT exclude_seat_time_conflict EXCLUDE USING gist (
        seat_id WITH =,
        daterange(start_date, end_date, '[]') WITH &&,
        int4range(start_minute, end_minute, '[)') WITH &&
```

```
    ) WHERE (status = 'ACTIVE'),

    -- MULTIDIMENSIONAL EXCLUSION CONSTRAINT 2: PREVENTS STUDENT TIME COLLISION
    CONSTRAINT exclude_student_time_conflict EXCLUDE USING gist (
        student_id WITH =,
        daterange(start_date, end_date, '[]') WITH &&,
        int4range(start_minute, end_minute, '[)') WITH &&
    ) WHERE (status = 'ACTIVE')
);

CREATE INDEX idx_memberships_student_id ON memberships(student_id);
CREATE INDEX idx_memberships_seat_id ON memberships(seat_id);
CREATE INDEX idx_memberships_dates ON memberships(start_date, end_date);
CREATE INDEX idx_memberships_status ON memberships(status);
CREATE INDEX idx_memberships_renewed_from ON memberships(renewed_from_id);

-- ============================================================================
-- 6. PAYMENTS (IMMUTABLE FINANCIAL LEDGER)
-- ============================================================================
CREATE TABLE payments (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    receipt_number VARCHAR(30) NOT NULL UNIQUE, -- RCP-YYYYMMDD-XXXX
    membership_id UUID NOT NULL REFERENCES memberships(id) ON DELETE RESTRICT,
    student_id UUID NOT NULL REFERENCES students(id) ON DELETE RESTRICT,

    amount NUMERIC(10, 2) NOT NULL CHECK (amount > 0),
    payment_method payment_method_enum NOT NULL,
    transaction_ref VARCHAR(100) NULL,          -- UPI UTR / Bank Reference
    status payment_status_enum NOT NULL DEFAULT 'COMPLETED',

    -- Voiding audit fields
    voided_at TIMESTAMPTZ NULL,
    voided_by UUID NULL REFERENCES users(id) ON DELETE SET NULL,
    void_reason TEXT NULL,

    notes TEXT NULL,
    received_by UUID NULL REFERENCES users(id) ON DELETE SET NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT chk_payment_void_meta CHECK (
        (status = 'VOIDED' AND voided_at IS NOT NULL AND void_reason IS NOT NULL)
        OR (status = 'COMPLETED' AND voided_at IS NULL AND void_reason IS NULL)
    )
);

CREATE INDEX idx_payments_membership_id ON payments(membership_id);
CREATE INDEX idx_payments_student_id ON payments(student_id);
CREATE INDEX idx_payments_status ON payments(status);
CREATE INDEX idx_payments_created_at ON payments(created_at);

-- ============================================================================
-- 7. ATTENDANCE LOGS
-- ============================================================================
CREATE TABLE attendance (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    student_id UUID NOT NULL REFERENCES students(id) ON DELETE RESTRICT,
    membership_id UUID NOT NULL REFERENCES memberships(id) ON DELETE RESTRICT,
    attendance_date DATE NOT NULL,

    check_in TIMESTAMPTZ NOT NULL,
    check_out TIMESTAMPTZ NULL,
    status attendance_status_enum NOT NULL DEFAULT 'PRESENT',
    source attendance_source_enum NOT NULL DEFAULT 'MANUAL_STAFF',
    marked_by UUID NULL REFERENCES users(id) ON DELETE SET NULL,
    notes TEXT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT chk_attendance_times CHECK (check_out IS NULL OR check_out >= check_in),
    CONSTRAINT uq_membership_daily_attendance UNIQUE (membership_id, attendance_date)
);

CREATE INDEX idx_attendance_student_date ON attendance(student_id, attendance_date);
```

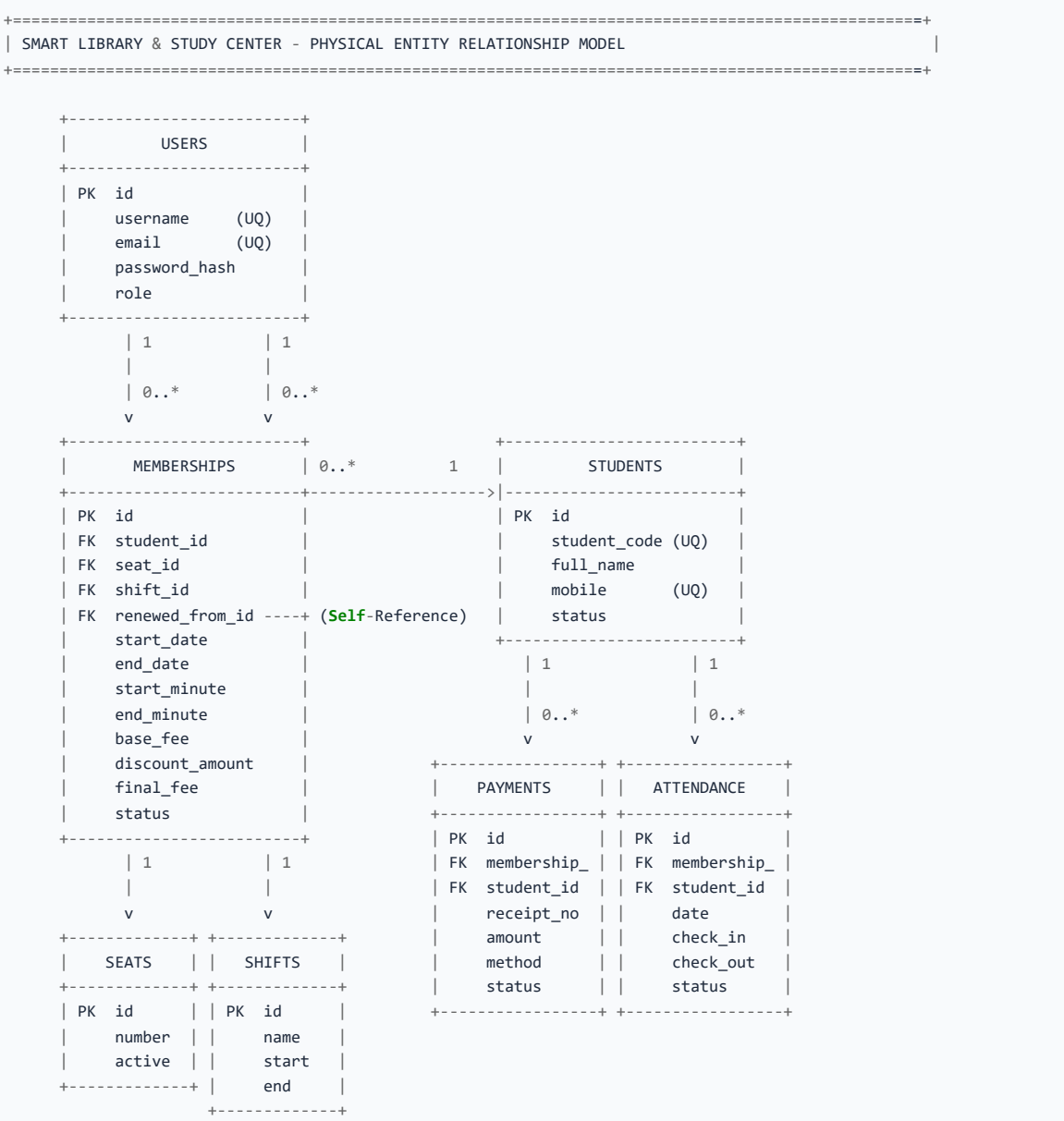
```sql
CREATE INDEX idx_attendance_date ON attendance(attendance_date);

-- ============================================================================
-- 8. SYSTEM AUDIT LOGS
-- ============================================================================
CREATE TABLE audit_logs (
    id BIGSERIAL PRIMARY KEY,
    user_id UUID NULL REFERENCES users(id) ON DELETE SET NULL,
    action VARCHAR(50) NOT NULL,
    entity_name VARCHAR(50) NOT NULL,
    entity_id VARCHAR(50) NOT NULL,
    old_values JSONB NULL,
    new_values JSONB NULL,
    ip_address VARCHAR(45) NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_audit_entity ON audit_logs(entity_name, entity_id);
CREATE INDEX idx_audit_created_at ON audit_logs(created_at);

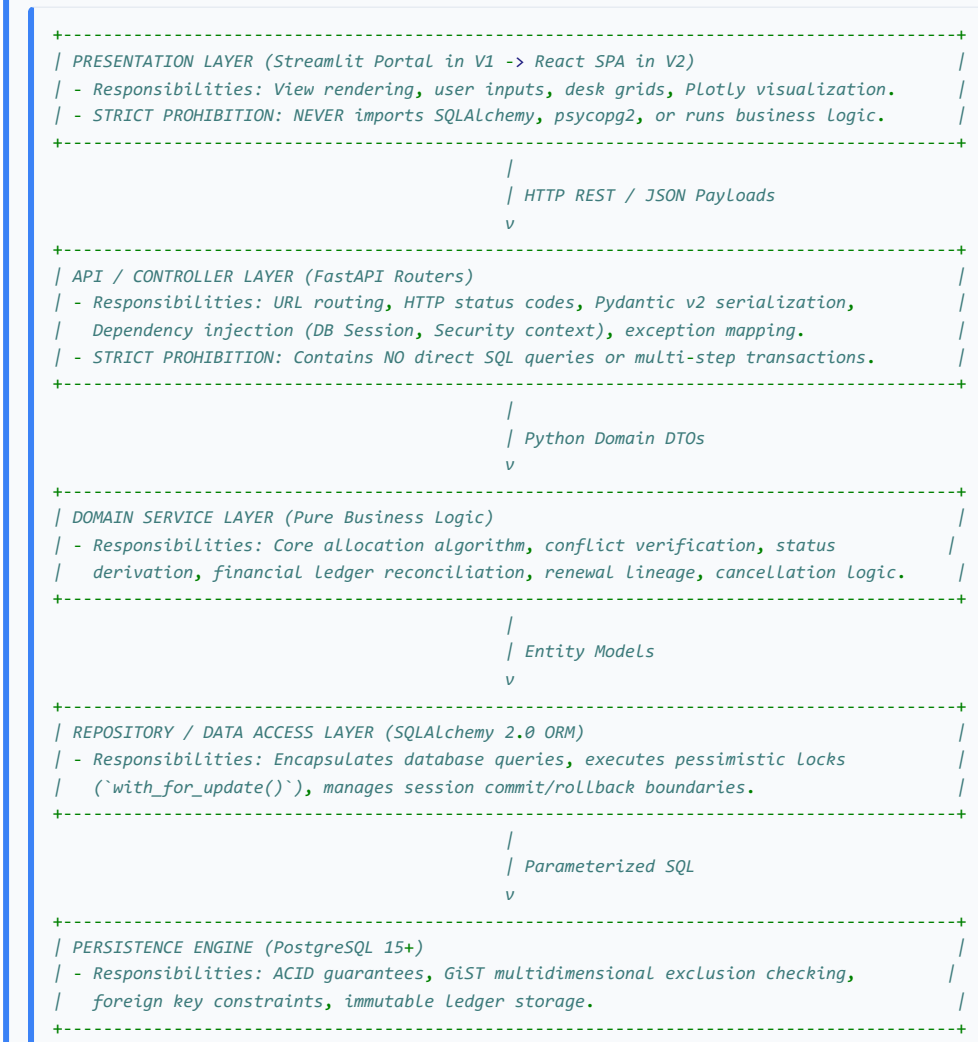
-- ============================================================================
-- 9. STANDARDIZED OPERATIONAL SQL VIEWS
-- ============================================================================
CREATE OR REPLACE VIEW v_memberships_expanded AS
SELECT
    m.id,
    m.student_id,
    st.full_name AS student_name,
    st.mobile AS student_mobile,
    m.seat_id,
    s.seat_number,
    m.shift_id,
    sh.name AS shift_name,
    m.start_date,
    m.end_date,
    m.final_fee,
    m.status AS persisted_status,
    -- Dynamic Derived Status
    CASE
        WHEN m.status = 'CANCELLED' THEN 'CANCELLED'
        WHEN CURRENT_DATE < m.start_date THEN 'UPCOMING'
        WHEN CURRENT_DATE > m.end_date THEN 'EXPIRED'
        WHEN CURRENT_DATE >= (m.end_date - INTERVAL '3 days') AND CURRENT_DATE <= m.end_date THEN 'EXPIRING_SOON'
        ELSE 'ACTIVE'
    END AS derived_status,
    -- Financial Balances
    COALESCE(SUM(p.amount) FILTER (WHERE p.status = 'COMPLETED'), 0.00) AS total_paid,
    m.final_fee - COALESCE(SUM(p.amount) FILTER (WHERE p.status = 'COMPLETED'), 0.00) AS due_amount,
    CASE
        WHEN COALESCE(SUM(p.amount) FILTER (WHERE p.status = 'COMPLETED'), 0.00) = 0 THEN 'PENDING'
        WHEN COALESCE(SUM(p.amount) FILTER (WHERE p.status = 'COMPLETED'), 0.00) < m.final_fee THEN 'PARTIAL'
        ELSE 'PAID'
    END AS payment_status
FROM memberships m
JOIN students st ON m.student_id = st.id
JOIN seats s ON m.seat_id = s.id
JOIN shifts sh ON m.shift_id = sh.id
LEFT JOIN payments p ON m.id = p.membership_id
GROUP BY m.id, st.id, s.id, sh.id;
```

8. Entity-Relationship (ER) Diagram



9. Application Architecture

The system enforces **Domain-Driven Layered Architecture (DDD Lite)** with strict boundaries between presentation, coordination, business logic, and database persistence.


PRESENTATION LAYER (Streamlit Portal in V1 -> React SPA in V2)
- Responsibilities: View rendering, user inputs, desk grids, Plotly visualization.
- STRICT PROHIBITION: NEVER imports SQLAlchemy, psycopg2, or runs business logic.
HTTP REST / JSON Payloads
API / CONTROLLER LAYER (FastAPI Routers)
- Responsibilities: URL routing, HTTP status codes, Pydantic v2 serialization,
Dependency injection (DB Session, Security context), exception mapping.
- STRICT PROHIBITION: Contains NO direct SQL queries or multi-step transactions.
Python Domain DTOs
DOMAIN SERVICE LAYER (Pure Business Logic)
- Responsibilities: Core allocation algorithm, conflict verification, status
derivation, financial ledger reconciliation, renewal lineage, cancellation logic.
Entity Models
REPOSITORY / DATA ACCESS LAYER (SQLAlchemy 2.0 ORM)
- Responsibilities: Encapsulates database queries, executes pessimistic locks
(`with_for_update()`), manages session commit/rollback boundaries.
Parameterized SQL
PERSISTENCE ENGINE (PostgreSQL 15+)
- Responsibilities: ACID guarantees, GiST multidimensional exclusion checking,
foreign key constraints, immutable ledger storage.

10. Folder Structure

```
library-management-system/
│
├── backend/
│   ├── app/
│   │   ├── api/
│   │   │   ├── v1/
│   │   │   │   ├── endpoints/
│   │   │   │   │   ├── auth.py             # Staff login & JWT issuance
│   │   │   │   │   ├── students.py         # Student registration & directory
│   │   │   │   │   ├── seats.py            # 100-Seat inventory & status toggles
│   │   │   │   │   ├── shifts.py           # Operating shift configurations
│   │   │   │   │   ├── memberships.py      # Admission, renewal, cancellation endpoints
│   │   │   │   │   ├── payments.py         # Ledger payments, receipts & voiding
│   │   │   │   │   ├── attendance.py       # Check-in and check-out logs
│   │   │   │   │   ├── dashboard.py        # Real-time KPIs & occupancy grids
│   │   │   │   │   └── reports.py          # Financial & utilization export APIs
│   │   │   │   └── router.py               # Aggregated API v1 router
│   │   │   │
│   │   ├── core/
│   │   │   ├── config.py                   # Pydantic BaseSettings (.env loading)
│   │   │   ├── security.py                 # Argon2id password hashing & JWT handling
│   │   │   ├── exceptions.py               # Domain business exceptions & HTTP mapping
│   │   │   └── logging.py                  # Structured JSON logging config
│   │   │
│   │   ├── db/
│   │   │   ├── session.py                  # SQLAlchemy engine & session maker
│   │   │   └── base.py                     # Declarative Base metadata for Alembic
│   │   │
│   │   ├── models/                         # SQLAlchemy 2.0 Typed Mapped ORM Models
│   │   │   ├── user.py
│   │   │   ├── student.py
│   │   │   ├── seat.py
│   │   │   ├── shift.py
│   │   │   ├── membership.py
│   │   │   ├── payment.py
│   │   │   ├── attendance.py
│   │   │   └── audit.py
│   │   │
│   │   ├── schemas/                        # Pydantic v2 DTOs (Request / Response)
│   │   │   ├── user.py
│   │   │   ├── student.py
│   │   │   ├── seat.py
│   │   │   ├── shift.py
│   │   │   ├── membership.py
│   │   │   ├── payment.py
│   │   │   ├── attendance.py
│   │   │   └── dashboard.py
│   │   │
│   │   ├── repositories/                   # Data Access Query Encapsulation
│   │   │   ├── base.py
│   │   │   ├── student_repository.py
│   │   │   ├── seat_repository.py
│   │   │   ├── membership_repository.py
│   │   │   ├── payment_repository.py
│   │   │   └── attendance_repository.py
│   │   │
│   │   ├── services/                       # Pure Domain Business Logic
│   │   │   ├── allocation_service.py       # Core seat conflict & booking engine
│   │   │   ├── membership_service.py       # Renewal, cancellation, status derivation
│   │   │   ├── payment_service.py          # Financial ledger & balance reconciliation
│   │   │   ├── attendance_service.py       # Check-in validation & time window verify
│   │   │   └── analytics_service.py        # Capacity, occupancy, and revenue analytics
│   │   │
│   │   ├── utils/                          # Time & date mathematical helpers
│   │   │   └── time_ranges.py
│   │   │
│   │   └── main.py                         # FastAPI ASGI Application Entrypoint
│   │
```

```
│   └── tests/
│       ├── conftest.py                    # Pytest fixtures & isolated DB session
│       ├── test_allocation_engine.py      # Conflict detection & boundary tests
│       ├── test_memberships.py            # Lifecycle, renewals & cancellation tests
│       ├── test_payments.py               # Ledger calculations, partial payments & voids
│       ├── test_concurrency.py            # Multithreaded race condition tests
│       └── test_api/                      # End-to-end API HTTP integration tests
│
├── frontend/
│   └── streamlit_app/
│       ├── app.py                         # Streamlit multipage application entrypoint
│       ├── api_client.py                  # Clean HTTP REST client calling FastAPI
│       ├── components/                    # UI widgets (Seat Grid 1-100, Stat cards)
│       └── pages/
│           ├── 1_Seats_Grid.py            # Interactive visual 100-desk occupancy map
│           ├── 2_Students.py              # Student registration & history directory
│           ├── 3_Memberships.py           # New admissions, renewals, cancellations
│           ├── 4_Payments.py              # Cashier desk: collect payments, view receipts
│           ├── 5_Attendance.py            # Daily check-in/check-out front desk
│           └── 6_Dashboard.py             # Real-time occupancy & revenue analytics
│
├── migrations/                            # Alembic Database Migrations
│   ├── env.py
│   ├── script.py.mako
│   └── versions/
│
├── scripts/
│   ├── seed_data.py                       # Seeds 100 physical seats, 4 shifts, admin user
│   ├── backup_db.sh                       # Automated pg_dump backup script
│   └── generate_pdf.py                    # PDF documentation compilation engine
│
├── docs/                                  # Master Architectural Specifications
│   └── MASTER_TECHNICAL_SPECIFICATION.md
│
├── .env.example                           # Environment variable configuration template
├── requirements.txt                       # Production Python dependencies
├── README.md                              # System overview and setup instructions
└── .gitignore
```

11. API Architecture & REST Endpoint Reference

11.1 Students API

Method	Endpoint	Request Body	Response (200/201)	Possible Errors
`POST`	`/api/v1/students`	`StudentCreateSchema`	`StudentResponseSchema`	`400` (Validation), `409` (`MOBILE_EXISTS`)
`GET`	`/api/v1/students`	Query: `search`, `status`, `skip`, `limit`	`Paginated[StudentResponseSchema]`	`400` (Bad Query)
`GET`	`/api/v1/students/{id}`	Path: `id` (UUID)	`StudentDetailResponseSchema`	`404` (`STUDENT_NOT_FOUND`)
`PATCH`	`/api/v1/students/{id}`	`StudentUpdateSchema`	`StudentResponseSchema`	`404`, `409` (`MOBILE_EXISTS`)

11.2 Seats & Shifts API

Method	Endpoint	Request Body	Response (200/201)	Possible Errors
`GET`	`/api/v1/seats`	Query: `is_active`	`List[SeatResponseSchema]`	None
`GET`	`/api/v1/seats/availability`	Query: `start_date`, `end_date`, `shift_id`	`SeatAvailabilityMatrixSchema`	`400` (Bad Dates), `404` (Shift Not Found)
`PATCH`	`/api/v1/seats/{id}/status`	`SeatStatusUpdateSchema` (`is_active`, `reason`)	`SeatResponseSchema`	`404`, `409` (Active Memberships Exist)
`GET`	`/api/v1/shifts`	None	`List[ShiftResponseSchema]`	None

11.3 Memberships API

Method	Endpoint	Request Body	Response (200/201)	Possible Errors
`POST`	`/api/v1/memberships`	`MembershipCreateSchema`	`MembershipResponseSchema`	`400`, `404`, `409` (`SEAT_ALREADY_BOOKED`, `STUDENT_TIME_COLLISION`)
`POST`	`/api/v1/memberships/{id}/renew`	`MembershipRenewSchema`	`MembershipResponseSchema`	`404`, `409` (`SEAT_ALREADY_BOOKED`), `400` (`CANNOT_RENEW_CANCELLED`)
`POST`	`/api/v1/memberships/{id}/cancel`	`MembershipCancelSchema` (`cancel_reason`)	`MembershipResponseSchema`	`404`, `400` (`ALREADY_CANCELLED`)
`GET`	`/api/v1/memberships`	Query: `seat_id`, `student_id`, `status`	`Paginated[MembershipDetailSchema]`	`400`
`GET`	`/api/v1/memberships/{id}`	Path: `id`	`MembershipDetailSchema`	`404` (`MEMBERSHIP_NOT_FOUND`)

11.4 Payments API

Method	Endpoint	Request Body	Response (200/201)	Possible Errors
`POST`	`/api/v1/payments`	`PaymentCreateSchema`	`PaymentResponseSchema`	`400` (`PAYMENT_EXCEEDS_DUE`), `404` (`MEMBERSHIP_NOT_FOUND`)
`POST`	`/api/v1/payments/{id}/void`	`PaymentVoidSchema` (`void_reason`)	`PaymentResponseSchema`	`403` (`ADMIN_ONLY`), `404`, `400` (`ALREADY_VOIDED`)
`GET`	`/api/v1/memberships/{id}/payments`	Path: `id`	`List[PaymentResponseSchema]`	`404`

11.5 Attendance API

Method	Endpoint	Request Body	Response (200/201)	Possible Errors
`POST`	`/api/v1/attendance/check-in`	`AttendanceCheckInSchema`	`AttendanceResponseSchema`	`400` (`NO_ACTIVE_MEMBERSHIP_TODAY`), `409` (`ALREADY_CHECKED_IN`)
`POST`	`/api/v1/attendance/{id}/check-out`	Path: `id`	`AttendanceResponseSchema`	`404`, `400` (`ALREADY_CHECKED_OUT`)
`GET`	`/api/v1/attendance/daily`	Query: `attendance_date`, `shift_id`	`List[AttendanceDetailSchema]`	`400`

11.6 Dashboard & Reports API

Method	Endpoint	Request Body	Response (200/201)	Possible Errors
`GET`	`/api/v1/dashboard/summary`	Query: `target_date`	`DashboardSummarySchema`	None
`GET`	`/api/v1/dashboard/occupancy-grid`	Query: `target_date`	`SeatGridMatrixSchema`	None
`GET`	`/api/v1/reports/expiring`	Query: `days_threshold` (default 3)	`List[MembershipExpirySchema]`	None
`GET`	`/api/v1/reports/revenue`	Query: `start_date` , `end_date`	`RevenueReportSchema`	`400`

---

12. Service & Business Logic Algorithms

12.1 Algorithm 1: Pure Domain Seat Conflict Detection

```python
def check_seat_availability(
    db_session: Session,
    seat_id: int,
    start_date: date,
    end_date: date,
    start_minute: int,
    end_minute: int,
    exclude_membership_id: Optional[UUID] = None
) -> Tuple[bool, Optional[Membership]]:
    """
    Evaluates whether a physical seat has conflicting active memberships.
    """
    seat = db_session.query(Seat).filter(Seat.id == seat_id).first()
    if not seat or not seat.is_active:
        return False, None

    query = db_session.query(Membership).filter(
        Membership.seat_id == seat_id,
        Membership.status == MembershipStatusEnum.ACTIVE,
        Membership.start_date <= end_date,
        Membership.end_date >= start_date,
        Membership.start_minute < end_minute,
        Membership.end_minute > start_minute
    )

    if exclude_membership_id:
        query = query.filter(Membership.id != exclude_membership_id)

    collision = query.first()
    if collision:
        return False, collision

    return True, None
```

12.2 Algorithm 2: Student Simultaneous Booking Conflict

```python
def check_student_time_conflict(
    db_session: Session,
    student_id: UUID,
    start_date: date,
    end_date: date,
    start_minute: int,
    end_minute: int,
    exclude_membership_id: Optional[UUID] = None
) -> Tuple[bool, Optional[Membership]]:
    """
    Enforces that a single student cannot hold overlapping shift hours.
    """
    query = db_session.query(Membership).filter(
        Membership.student_id == student_id,
        Membership.status == MembershipStatusEnum.ACTIVE,
        Membership.start_date <= end_date,
        Membership.end_date >= start_date,
        Membership.start_minute < end_minute,
        Membership.end_minute > start_minute
    )

    if exclude_membership_id:
        query = query.filter(Membership.id != exclude_membership_id)

    collision = query.first()
    if collision:
        return False, collision

    return True, None
```

12.3 Algorithm 3: Atomic Admission with Pessimistic Locking

```python
def create_membership_atomic(
    db_session: Session,
    student_id: UUID,
    seat_id: int,
    shift_id: int,
    start_date: date,
    end_date: date,
    discount_amount: Decimal,
    initial_payment: Optional[PaymentCreateInput],
    creator_user_id: UUID
) -> Membership:
    """
    Executes student admission within an isolated ACID transaction.
    """
    try:
        # Step 1: Pessimistic Row Lock on Seat
        seat = db_session.query(Seat).filter(Seat.id == seat_id).with_for_update().first()
        if not seat:
            raise NotFoundException("SEAT_NOT_FOUND", f"Seat {seat_id} not found.")
        if not seat.is_active:
            raise ValidationException("SEAT_INACTIVE", f"Seat {seat_id} is under maintenance.")

        # Step 2: Fetch Shift Template
        shift = db_session.query(Shift).filter(Shift.id == shift_id).first()
        if not shift or not shift.is_active:
            raise NotFoundException("SHIFT_NOT_FOUND", "Invalid shift configuration.")

        # Step 3: Conflict Checks
        free, seat_conflict = check_seat_availability(
            db_session, seat_id, start_date, end_date, shift.start_minute, shift.end_minute
        )
        if not free:
            raise ConflictException(
                "SEAT_ALREADY_BOOKED",
                f"Seat {seat_id} is already occupied by Student {seat_conflict.student_id}."
            )

        student_free, student_conflict = check_student_time_conflict(
            db_session, student_id, start_date, end_date, shift.start_minute, shift.end_minute
        )
        if not student_free:
            raise ConflictException(
                "STUDENT_TIME_COLLISION",
                f"Student already has an active allocation on Seat {student_conflict.seat_id}."
            )

        # Step 4: Billing
        final_fee = shift.base_fee - discount_amount
        if final_fee < 0:
            raise ValidationException("INVALID_DISCOUNT", "Discount exceeds base fee.")

        # Step 5: Persist Membership
        membership = Membership(
            id=uuid4(),
            student_id=student_id,
            seat_id=seat_id,
            shift_id=shift_id,
            start_date=start_date,
            end_date=end_date,
            start_minute=shift.start_minute,
            end_minute=shift.end_minute,
            base_fee=shift.base_fee,
            discount_amount=discount_amount,
            final_fee=final_fee,
            status=MembershipStatusEnum.ACTIVE,
            created_by=creator_user_id
        )
        db_session.add(membership)
```

```
        db_session.flush()

        # Step 6: Initial Payment
        if initial_payment and initial_payment.amount > 0:
            if initial_payment.amount > final_fee:
                raise ValidationException("OVERPAYMENT", "Payment exceeds membership fee.")
            payment = Payment(
                id=uuid4(),
                receipt_number=generate_receipt_number(),
                membership_id=membership.id,
                student_id=student_id,
                amount=initial_payment.amount,
                payment_method=initial_payment.payment_method,
                transaction_ref=initial_payment.transaction_ref,
                status=PaymentStatusEnum.COMPLETED,
                received_by=creator_user_id
            )
            db_session.add(payment)

        db_session.commit()
        db_session.refresh(membership)
        return membership

    except Exception as e:
        db_session.rollback()
        raise e
```

---

13. Payment System & Financial Ledger

13.1 Single Source of Truth Financial Design

- **Fee Target Contract**: Recorded on `memberships` (`final_fee = base_fee - discount_amount`).
- **Ledger Entries**: Recorded on `payments` . Each payment row is an immutable financial transaction.
- **Computed Balances**:

$$\text{Total Paid} = \sum_{p \in \text{payments}, p.\text{status}='\text{COMPLETED}'} p.\text{amount}$$

\text{Due Balance} = \max(0, \text{final_fee} - \text{Total Paid})

- **Payment Statuses**:
  - `PAID` : \text{Total Paid} \ge \text{final_fee}
  - `PARTIAL` : 0 < \text{Total Paid} < \text{final_fee}
  - `PENDING` : $\text{Total Paid} = 0$

13.2 Voiding Workflow & Security Invariants

- Direct row deletion (`DELETE FROM payments`) is strictly prohibited.
- Voiding is restricted exclusively to users holding role `ADMIN` .
- Allowed strictly within 24 hours of creation, accompanied by a mandatory `void_reason` .
- Marking a row `status = 'VOIDED'` automatically restores the student's due balance on all queries.

---

14. Renewal System & Chain Lineage

14.1 The Renewal Contract

Renewal creates a **new membership row** while preserving the historical membership record:

- `renewed_from_id = old_membership.id` establishes the audit lineage.

- **Start Date**: Typically `old_membership.end_date + 1 day`. If renewed after an absence, start date is the rejoining date.
- **Seat Retention**: The student may renew the same seat or migrate to a new seat.
- **Shift Migration**: The student may retain their shift or switch shifts.
- **Pre-Expiry Renewal**: Permitted. Because conflict detection evaluates the target future date range, the unexpired current membership does not block the future renewal.
- **Outstanding Dues**: Dues on prior memberships remain attached to the prior membership row, but the student directory profile alerts staff of historical debt.

---

15. Attendance System & Check-in Rules

15.1 Check-In Pipeline

1. Scan student code or mobile number.
2. Locate membership where: `status == 'ACTIVE'` and `start_date <= CURRENT_DATE <= end_date`.
3. Verify operating window: Check-in allowed within shift window $\pm 30\ \text{minutes}$ grace threshold.
4. Verify daily uniqueness: Enforced by `UNIQUE (membership_id, attendance_date)`.
5. Check-out marks departure time and calculates total hours studied.

---

16. Dashboard, Metrics & Analytics Engine

16.1 Solving the 100-Seat Capacity Paradox

Because 100 physical seats are time-multiplexed across shifts, measuring capacity as $\frac{\text{Active Students}}{100}$ produces misleading results. The system computes three distinct KPIs:

1. Physical Seat Utilization Rate

$$\text{Physical Utilization Rate} = \frac{|s \in \text{Seats} \mid \text{seat } s \text{ has active booking today}|}{100} \times 100$$

2. Slot-Capacity Occupancy Rate (Shift-Slot Occupancy)

$$\text{Total Daily Slot Capacity} = 100 \text{ physical seats} \times 3 \text{ standard shifts} = 300 \text{ shift-slots}$$

- `MORNING`, `AFTERNOON`, `EVENING` memberships consume **1 slot**.
- `FULL_DAY` memberships consume **3 slots**.

$$\text{Slot Occupancy Rate} = \frac{\sum_{m \in \text{ActiveMembershipsToday}} \text{SlotWeight}(m.\text{shift})}{300} \times 100$$

3. Seat-Hour Capacity Occupancy Rate (Time-Weighted)

$$\text{Total Daily Facility Capacity} = 100 \text{ seats} \times 16 \text{ operating hours} = 1,600 \text{ seat-hours/day}$$

$$\text{Seat-Hour Occupancy Rate} = \frac{\sum_{m \in \text{ActiveMembershipsToday}} \text{DurationHours}(m.\text{shift})}{1,600} \times 100$$

Where:

- Morning = 4 hours
- Afternoon = 4 hours
- Evening = 8 hours
- Full Day = 16 hours

---

17. Security Architecture & Threat Mitigation

17.1 Security Blueprint

- **Password Hashing**: **Argon2id** (memory cost 64MB, time cost 3 iterations).
- **Stateless Authentication**: JWT Bearer tokens signed with HMAC-SHA256, 8-hour expiry.
- **Role-Based Access Control (RBAC)**: Enforced via FastAPI dependencies (`require_role(ADMIN)`).
- **SQL Injection Prevention**: 100% parameterized queries via SQLAlchemy 2.0 ORM.
- **Audit Logging**: Sensitive mutations (voids, cancellations) written to `audit_logs` .

18. Testing Strategy & Pytest Verification Matrix

```
+-----------------------------------------------------------------------------------------+
| TEST SCENARIOS FOR SEAT ALLOCATION & CONFLICT ENGINE                                    |
+-----------------------------------------------------------------------------------------+
| Test ID | Scenario Description               | Parameters / Shifts         | Expected Result |
+---------+------------------------------------+-----------------------------+-----------------+
| TC-001  | Non-overlapping consecutive shifts | Seat 25: Morning (07-11)    | ALLOW (201)     |
|         | on same physical seat              | Seat 25: Afternoon (11-15)  |                 |
+---------+------------------------------------+-----------------------------+-----------------+
| TC-002  | Non-overlapping afternoon & evening| Seat 25: Afternoon (11-15)  | ALLOW (201)     |
|         | on same physical seat              | Seat 25: Evening (15-23)    |                 |
+---------+------------------------------------+-----------------------------+-----------------+
| TC-003  | Touching boundary condition        | Shift 1 ends exactly 11:00  | ALLOW (201)     |
|         | (11:00 boundary handover)          | Shift 2 starts exactly 11:00|                 |
+---------+------------------------------------+-----------------------------+-----------------+
| TC-004  | Partial overlap collision          | Seat 25: Morning (07-11)    | REJECT (409)    |
|         |                                    | Seat 25: Custom (09-13)     |                 |
+---------+------------------------------------+-----------------------------+-----------------+
| TC-005  | Full Day engulfing Morning shift   | Seat 25: Full Day (07-23)   | REJECT (409)    |
|         | on same date                       | Seat 25: Morning (07-11)    |                 |
+---------+------------------------------------+-----------------------------+-----------------+
| TC-006  | Full Day engulfing Evening shift   | Seat 25: Full Day (07-23)   | REJECT (409)    |
|         | on same date                       | Seat 25: Evening (15-23)    |                 |
+---------+------------------------------------+-----------------------------+-----------------+
| TC-007  | Identical shift on different seats | Seat 25: Morning (07-11)    | ALLOW (201)     |
|         | (Different chairs)                 | Seat 26: Morning (07-11)    |                 |
+---------+------------------------------------+-----------------------------+-----------------+
| TC-008  | Student double booking collision   | Rahul: Seat 10 (07-11)      | REJECT (409)    |
|         | (Same student, 2 desks, same time) | Rahul: Seat 50 (07-11)      |                 |
+---------+------------------------------------+-----------------------------+-----------------+
| TC-009  | Student multi-shift allowed        | Rahul: Seat 10 (07-11)      | ALLOW (201)     |
|         | (Same student, non-overlapping)    | Rahul: Seat 50 (15-23)      |                 |
+---------+------------------------------------+-----------------------------+-----------------+
| TC-010  | Expired membership seat re-booking | Student A: Oct 1 - Oct 31   | ALLOW (201)     |
|         | (Subsequent month booking)         | Student B: Nov 1 - Nov 30   |                 |
+---------+------------------------------------+-----------------------------+-----------------+
| TC-011  | Cancelled membership immediate free| Student A cancels on Oct 5  | ALLOW (201)     |
|         |                                    | Student B books Oct 6-31    |                 |
+---------+------------------------------------+-----------------------------+-----------------+
| TC-012  | Concurrent booking race condition  | Two parallel worker threads | Exactly 1 WINS  |
|         | targeting exact same seat & shift  | requesting Seat 1 simultaneously| 1 REJECTS (409) |
+-----------------------------------------------------------------------------------------+
```

19. Real-World Edge Cases & Handling Matrix

Edge Case Scenario	Defensive System Behavior
Duplicate Mobile Number	Rejected by unique index. Returns `409 Conflict: MOBILE_ALREADY_EXISTS`. Directs staff to existing profile.
Early Renewal Before Expiry	Allowed. Start date set to `old.end_date + 1 day`. Conflict engine checks future dates; succeeds cleanly.
Late Renewal After Expiry Gap	Allowed. Start date set to current date. Renewal lineage linked via `renewed_from_id`.
Seat Change on Renewal	Allowed. System validates availability of target seat for the renewal period. Prior history remains on old seat.
Shift Change on Renewal	Allowed. System validates new shift hours against target seat availability.
Partial Payment Received	Recorded as `COMPLETED` payment. Membership derives `status = 'PARTIAL'`, with `due_amount > 0`.
Overpayment Attempt	Service layer compares amount against due balance. Rejects with `400 Bad Request: PAYMENT_EXCEEDS_DUE`.
Voiding Settled Payment	Permitted only for `ADMIN` within 24 hours with mandatory `void_reason`. Restores due balance immediately.
Broken / Maintenance Desk	Admin marks seat `is_active = FALSE`. Blocks future allocations while retaining historical audit records.
Attendance on Expired Subscription	Rejected at check-in. Returns `400 Bad Request: NO_ACTIVE_MEMBERSHIP_TODAY`.
Concurrent Staff Allocations	Serialized via `SELECT ... FOR UPDATE` and backstopped by PostgreSQL GiST exclusion constraint. Zero double-bookings.

20. Backup, Disaster Recovery & Data Portability

20.1 Automated PostgreSQL Backup Script

```bash
#!/bin/bash
# scripts/backup_db.sh - Automated PostgreSQL Daily Backup
set -e

BACKUP_DIR="/var/backups/postgres/library_db"
TIMESTAMP=$(date +"%Y%m%d_%H%M%S")
FILENAME="$BACKUP_DIR/library_backup_$TIMESTAMP.sql.gz"

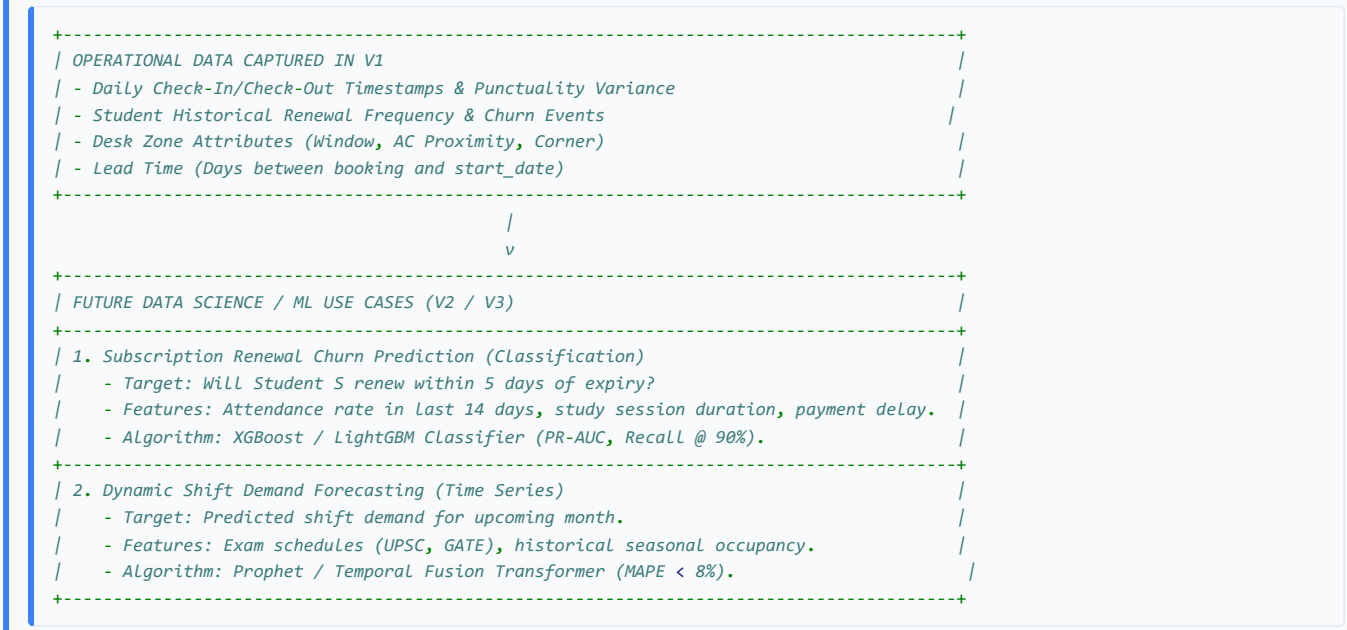
mkdir -p "$BACKUP_DIR"
pg_dump -U postgres -h localhost -d library_db | gzip > "$FILENAME"
find "$BACKUP_DIR" -type f -name "*.sql.gz" -mtime +30 -exec rm {} \;

echo "Backup successful: $FILENAME"
```

20.2 Data Portability

- **CSV / Excel Export**: Endpoints `/api/v1/reports/revenue/export` and `/api/v1/reports/students/export` stream clean tabular CSV data for third-party accounting.

21. Future Data Science & Machine Learning Architecture



22. Phased Development Roadmap

Implementation is divided into **14 discrete, testable phases**:

- **Phase 01: Architecture & Technical Specification Approval** (Completed)
- **Phase 02: PostgreSQL Database Engine Setup & Alembic Migrations**
- **Phase 03: FastAPI Core Foundation, Logging, Exception Handlers & DB Session**
- **Phase 04: Master Repositories: Students, Seats (1–100), and Shifts**
- **Phase 05: Seat Allocation Engine & Multidimensional Conflict Resolution**
- **Phase 06: Membership Lifecycle Engine (Admissions, Validations)**
- **Phase 07: Financial Ledger & Multi-Payment Processing**
- **Phase 08: Subscription Renewal Engine & Chain Preservation**
- **Phase 09: Cancellation Workflow & Immediate Seat Release**
- **Phase 10: Attendance Check-In / Check-Out Tracking**
- **Phase 11: End-to-End Pytest Automated Testing Suite**
- **Phase 12: Streamlit Interactive Front-Desk Application**
- **Phase 13: Executive Analytics, Capacity KPIs & Financial Reporting**
- **Phase 14: Authentication, RBAC Security & Production Hardening**

23. Architecture Decisions and Rationale (ADRs)

- **ADR-001 (Derived vs Persisted Status)**: Persist only `ACTIVE` and `CANCELLED`. Derive `EXPIRING_SOON` and `EXPIRED` dynamically to prevent stale data bugs and table write bloat.
- **ADR-002 (Half-Open Time Intervals)**: Shifts defined as $[T_{\text{start}}, T_{\text{end}})$ in discrete minutes to permit touching boundary handover without collision.
- **ADR-003 (Inclusive Calendar Days)**: Memberships defined as $[D_{\text{start}}, D_{\text{end}}]$ inclusive to align with commercial library subscription norms.
- **ADR-004 (Two-Tier Concurrency Control)**: Pessimistic locking (`FOR UPDATE`) paired with GiST exclusion constraints for mathematical double-booking prevention.

- **ADR-005 (Immutable Financial Ledger)**: Billing obligations separated from payment ledger records to maintain accounting integrity and support partial payments.

---

24. Known Limitations of Version 1.0

1. **Single Facility Scope**: Built for a single branch with 100 seats. Multi-branch support scheduled for V2.
2. **Fixed Canonical Shifts**: Operates on 4 defined shifts; arbitrary hourly micro-bookings are unsupported.
3. **Manual Refunds**: Cancellations free seats immediately, but cash/UPI refunds are handled manually outside the system.
4. **Manual & Web Kiosk Attendance**: Physical biometric scanner direct TCP sync is deferred to V2.

---

25. Future Expansion & Enterprise Roadmap

1. **React / Next.js Admin & Student Portal**: Upgrading frontend to modern responsive web application.
2. **Student Mobile Application (Flutter)**: Enabling seat browsing, UPI payments, and digital ID cards.
3. **Automated Biometric Turnstile Integration**: Direct TCP socket worker integration with ZKTeco hardware.
4. **Machine Learning Renewal Risk Alerts**: Automated notification queue flagging at-risk churn students.
5. **Multi-Branch Multi-Tenant SaaS**: Tenant isolation for multi-facility library operators.

MASTER ARCHITECTURE & SPECIFICATION COMPLETED AND APPROVED